

PV-PP GETTING STARTED

V2.1.167 PRODUCTIVE VALUE-PRODUCTIVE POWER FRAMEWORK



Lance Amundsen | amundsenlance@gmail.com
<https://amundsenlance.github.io/pvpp-runtime-api/>

PV-PP Getting Started

Edition 2.1, for PV-PP Runtime v2.1.167 (Framework 2.1, Build 167). 2026.

DOI: 10.5281/zenodo.23238468. Previous version, for runtime build 141: 10.5281/zenodo.23071376.

Copyright © 2026 Lance Amundsen. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0): <https://creativecommons.org/licenses/by/4.0/>

PV-PP™ is a trademark of Lance Amundsen. The license covers the text of this book; it grants no rights in the trademark.

The tutorial programs in Chapters 5 and 6 and the rules and prompts in Chapter 8 are also distributed in the PV-PP runtime repository and are licensed there under the Apache License, Version 2.0, the license of that repository. They are provided “as is”, without warranties or conditions of any kind.

Companion volumes: PV-PP Programmer’s Guide and PV-PP Programmer’s Reference. Their editions for runtime v2.1.167 follow this one; their editions for runtime build 141 are 10.5281/zenodo.23072420 and 10.5281/zenodo.23072506.

Runtime, companion files, and all three volumes: <https://github.com/AmundsenLance/pvpp-runtime-api> and <https://amundsenlance.github.io/pvpp-runtime-api/>

Contact: Lance Amundsen, amundsenlance@gmail.com

About This Book

This is the shortest way into the Productive Value–Productive Power (PV-PP) runtime. It explains what the PV-PP framework is and what the runtime does, recommends building with an AI assistant, shows how to install and verify the runtime, and then builds one small application, a battery-powered sensor, in five runnable steps, with a sixth that stops the sensor’s recharge when its permission is withdrawn while it is under way. It is the first of three volumes. The PV-PP Programmer’s Guide teaches the framework and application design at length, and the PV-PP Programmer’s Reference states exactly what the runtime provides. Read this book first. You do not need the other two to finish it.

Which runtime. Everything here describes PV-PP Runtime v2.1.167: Framework Version 2.1, runtime build 167, frozen. The build was developed under the identifier v0.167, and its package metadata still reports version 0.167; both names refer to the same frozen files. The runtime is in the folder runtime-v2.1.167 of the repository, beside the folders of the earlier runtimes. Section 3.2 shows how to prove that the copy you have is that build.

What changed from the previous edition. The previous edition described build 141. Build 167 changes nothing in the framework and nothing in the runtime’s decision core: 205 of the 206 files in build 141’s manifest are byte-identical in build 167, none is missing, and the one that differs, pyproject.toml, carries only the new package metadata. Everything the previous edition said still holds; the supervision layer lives in new, separate files. Build 167 adds a supervision layer that governs an action while it runs (Section 2.3; Chapter 6). This edition also states the rule for what a new actor inherits from the one that created it (Section 1.4), which the framework already implied but the previous edition did not say. Section 2.5 shows, on one page, what you can do with the PV-PP framework: develop, validate, and govern agents.

The book stands alone. Everything you need to read is printed here: the tutorial programs and their output in Chapters 5 and 6, and the rules and prompts for AI assistants in Chapter 8. Every listing in Chapters 5 and 6 is reproduced verbatim from files that run against the unmodified, hash-verified runtime; every output shown is what those files print, and the thirteen tests in Sections 5.7 and 6.6 pass.

Companion files. So that you do not have to retype anything, the repository’s getting-started folder holds exact copies of that material: the eight tutorial files in getting-started/battery, to run, and Listings 8.1 to 8.4 in getting-started/ai-rules-and-prompts.txt, to paste into an assistant. Neither contains anything the book does not.

Citations. A citation such as Guide 1.9 or Reference 3.1 names a section of one of the other two volumes, which the opening of Chapter 1 introduces.

Contents

About This Book.....	2
Chapter 1 — What the PV-PP Framework Is.....	5
1.1 Productive Value and Productive Power.....	5
1.2 Why Not One Score.....	6
1.3 Floors, Ceilings, and Corridors.....	6
1.4 What a Copy Inherits.....	7
1.5 Build It with an AI Assistant.....	7
1.6 How This Book Is Organized.....	8
Chapter 2 — What the Runtime Does.....	9
2.1 Who Does What.....	9
2.2 The Stages.....	9
2.3 What Build 167 Adds.....	10
2.4 What the Runtime Does Not Do.....	11
2.5 What You Can Do with the PV-PP Framework.....	12
Chapter 3 — Installing and Verifying the Runtime.....	13
3.1 What You Need.....	13
3.2 Verify That You Have the Frozen Build.....	13
3.3 Keep Your Code Outside the Runtime.....	13
Chapter 4 — The Calls You Actually Need.....	15
4.1 What to Ignore at First.....	16
4.2 When a Call Refuses.....	16
Chapter 5 — Tutorial: A Battery-Powered Sensor.....	18
5.1 Step 1: Declare What Exists.....	18
5.2 Step 2: Tell the Runtime What the World Looks Like.....	20
5.3 Step 3: Say What Each Option Would Do.....	21
5.4 Step 4: Run One Decision Cycle.....	22
5.5 Step 5: Carry Out the Decision.....	24
5.6 What You Just Used.....	25
5.7 Test It.....	25
5.8 Where to Go Next.....	26
Chapter 6 — Tutorial, Continued: Stopping an Action That Loses Its Authority.....	28
6.1 What Supervision Adds.....	28
6.2 The Situation.....	29
6.3 Step 6: Supervise the Recharge.....	29
6.4 Reading the Output.....	33
6.5 When the Withdrawal Arrives Matters.....	33
6.6 Test It.....	34
6.7 What You Just Used.....	35
Chapter 7 — Seven Things the Runtime Will Not Do for You.....	37

Chapter 8 — Developing with AI.....	39
8.1 Why AI Helps Here.....	39
8.2 Where AI Goes Wrong with the PV-PP Runtime.....	39
8.3 How to Work.....	40
8.4 The Rules to Paste into Every Prompt.....	41
8.5 Three Prompts.....	41
8.6 What Stays with You.....	42
Chapter 9 — Where to Go Next.....	43
9.1 Code to Read Next.....	43
9.2 Working with an Assistant.....	44

Chapter 1 — What the PV-PP Framework Is

The Productive Value–Productive Power (PV-PP) framework describes productive systems and governs the decisions made within them. A productive system is anything whose ability to keep acting depends on the state it is in: a person, a household, a machine, an organization, or a software agent, together with the resources, capabilities, information, rules, and possible changes that make its actions possible.

The PV-PP runtime is software that puts the framework to work. It is a Python package that holds a represented model of a productive system (its state, its capabilities, the rules that bind it, and the options open to it) and governs the decisions made about it: which course of action to take, whether that action is licensed for execution, and whether the new state the host reports is a valid result of that licensed execution. The runtime does not replace the framework. It implements a governed part of it, and the Programmer’s Reference states exactly which part (Guide 1.5; Reference Chapter 15).

This book is one of a three-document set:

- **PV-PP Getting Started**, this book: what the PV-PP framework and runtime are, how to install and verify the runtime, a first application and how to supervise it while it acts, and how to build with an AI assistant.
- **PV-PP Programmer’s Guide**, the Guide: the framework itself, how to model a productive system, and how to design, build, test, and supervise an application, at length and with worked examples.
- **PV-PP Programmer’s Reference**, the Reference: exactly what runtime v2.1.167 provides, call by call, with what each call accepts, returns, refuses, and leaves to you.

In this book, “the Guide” and “the Reference” mean the other two volumes, and a citation such as Guide 1.9 or Reference 3.1 names a section in one of them. This chapter condenses the Guide’s opening chapter to what a programmer needs before starting; the Guide’s Chapter 1 gives the full account.

1.1 Productive Value and Productive Power

Many decision systems reduce every option to a single number, such as a score, a utility, a reward, or a weighted sum of costs and benefits, and choose the option with the largest number. This is called scalar aggregation. It is simple and often works, but it lets a gain on one concern make up for a loss on another, and it merges questions that need separate answers. The PV-PP framework starts by separating two of them. What passes between parties when they interact? And what are they able to do afterward? Productive Value (PV) is the value actually conveyed through interaction: goods, services, money, information, care, and harmful effects as well as beneficial ones. Productive Power (PP) is actual capability, under stated conditions, to produce or preserve that value.

The two can move in different directions. An interaction can deliver something valuable now while damaging what remains possible later, or impose a cost now while building capability. The PV-PP framework also separates actual capability from perceived capability, what an actor believes it can do, because decisions are made from what is perceived. The runtime keeps the host’s record of actual state and the perceived state the decision is made on as separate objects (Chapter 5, step 2).

In the tutorial of Chapter 5, the sensor's measurements are the value it produces, and the energy in its battery is the productive power that makes them possible. Every measurement spends some of that power.

1.2 Why Not One Score

The PV-PP framework does not reject scalar aggregation. Its scalar-reduction proof program establishes where a single score can stand in for the framework's staged selection and where it cannot, and both results carry conditions (Guide 1.7; Guide 11.2–11.3):

- **Containment.** On a precisely defined class of decisions, roughly those in which every option's consequences lie along a single line and moving along that line never makes any governing concern worse and improves at least one, a plain unweighted average picks out exactly the same set of best options as the first stage of the framework's selection. The framework can check from its own objects whether a decision belongs to that class.
- **Non-reduction.** When at least two governing concerns genuinely conflict, and options that tie on them still differ in other respects, no score of any form can faithfully reproduce the staged selection: a score that tries is forced to merge stages the framework keeps separate.

Neither result is universal. Containment is proved for one class of decisions, not for scoring in general, and non-reduction holds only under its stated conditions; it does not show that every scoring method fails. The PV-PP framework keeps different failures apart: a missing option is not a forbidden one, a forbidden option is not an inadequate one, and a selected option is not yet an authorized action.

1.3 Floors, Ceilings, and Corridors

Viable operation has two sides. A lower bound, or floor, is a minimum the system needs to keep going: enough reserve, capability, evidence, safety, or recovery capacity. An upper bound, or ceiling, is a limit it must not exceed: more exposure, reachability, resource use, or authority than it may safely or legitimately hold. More capability is not always better. An autonomous agent that acquires access beyond what its authority covers has crossed a ceiling, even though it can now do more (Guide 1.9).

A corridor is the region of states and paths that keeps every floor and ceiling satisfied together. It is a way of describing the result of the framework's existing constraints, adequacy, recovery, and authority machinery, not an additional mechanism (Guide 1.9.1).

In the runtime the two sides are expressed differently. A registered domain's threshold is a floor: the runtime measures a domain's margin as its value minus its threshold, and the domain fails when the margin is used up. The tutorial's battery, which fails at 2.0 units, is a floor. Ceilings are enforced mainly as hard rules at the Constraints stage, and through execution authority: a license authorizes only the decision the runtime selected, and when the host reports a change in the facts that authority depends on, the runtime works out which stages must be governed again (Reference Chapters 11 and 12). The Guide's operations-agent example uses both (Guide 10.5). A limit that is better modeled as a domain can still be written as a floor, by reporting the headroom that remains below the ceiling as the domain's value. Build 167 adds one more route for ceilings. When a monitoring tool reports, while an action is under way, that an authority the action depends on has been withdrawn, the runtime can decide, at a checkpoint your code calls, that the action may not continue; your code then issues a stop request, which the runtime validates and records, and your controller carries it out (Chapter 6).

1.4 What a Copy Inherits

Productive systems create other productive systems. An organization opens a branch, a fleet commissions a replacement unit, and a software agent starts a sub-agent or is copied into a new run. The PV-PP framework is strict about what passes from the one that creates to the one created.

- **Configuration and information can be inherited.** A design, settings, a prompt, code, tools, a record of what worked.
- **Permissions and standing state are inherited only when declared.** Your application must declare the transfer explicitly. Otherwise the new actor starts without them.
- **Current Productive Power is never inherited.** Productive Power is actual capability under stated conditions (Section 1.1), and the new actor's conditions are its own. Until its capability has been established from evidence about the new actor itself, by measurement, qualification, or observation, nothing that depends on its Productive Power may rely on it.

The reason is practical. A new agent that receives a calculator tool and permission to use it has not thereby acquired the ability to do arithmetic: the interface may fail, the model may choose the wrong tool, the permission may be withdrawn. Inherited configuration is a possible basis for capability, not the capability. Controlled experiments with populations of AI agents show why the line matters: strongly inherited configuration did not, by itself, show inherited capability (Amundsen, Before AI Agents Evolve in the Wild, Version 2.0, 2026, DOI 10.5281/zenodo.23175664).

In the runtime this is a duty of the host, not a check the runtime makes. When your code creates a new actor from an existing one, carry over the definitions you choose into its registry, and form its perceived state from evidence about the new actor. Never copy the parent's ProductivePowerState values into the child. If the sensor of Chapter 5 were replaced by a new unit built to the same design, the new unit would inherit the registry—the domain, its threshold, the actions, and the rules—but its battery level would come from measuring its own battery. This is not a change to the framework, and build 167 does not enforce it; it states how an application must handle inheritance under the framework as it stands.

1.5 Build It with an AI Assistant

For any PV-PP application beyond a small one, develop with an AI assistant. The runtime does not require it, but in practice it is close to a necessity. The runtime exposes several hundred names, and the three volumes that document them run to more than 500 pages. No one should have to hold that in their head to write a small application, and an assistant with the Reference attached can find the call you need, what it checks, and what it leaves to you, in seconds. The framework itself was formalized, implemented, and tested with substantial AI assistance, and the Guide recommends the same approach (Guide 1.12).

Assistants also make confident mistakes, and the PV-PP runtime has specific traps for them: legacy entry points that still work, a projection route that skips the runtime's checks, duties the runtime cannot enforce, and a supervision layer that is easy to describe as stopping processes itself, which it does not. Chapter 8 turns those traps into a working method, supplied ready to use:

- **Rules to paste into every conversation** (Listing 8.1). Nine blunt rules that keep an assistant on the supported API and the frozen runtime, and make it cite the Reference for every call.
- **Prompt 1, model my situation** (Listing 8.2). The assistant interviews you, one question at a time, until it can write a definition of your model, and lists every number it had to assume.

- **Prompt 2, write the host code** (Listing 8.3). It generates your application in the shape of the tutorial, with tests, and first shows you every runtime call it will use and where the Reference documents it.
- **Prompt 3, review this host code** (Listing 8.4). A hostile review, ideally by a different assistant, against the Reference and the rules.

The Guide adds an interview prompt for each of its chapters (Guide Appendix A). Work through the tutorial in Chapters 5 and 6 yourself even if an assistant will write your code: it is the pattern the assistant is told to follow, and you need to recognize that pattern to review what the assistant produces.

Let the AI read and draft. Keep the decisions, the facts, and the verification for yourself.

1.6 How This Book Is Organized

Table 1.1 — The chapters of this book

Chapter	What it gives you
2 What the Runtime Does	Who does what, the stages of a decision, what build 167 adds, what the runtime leaves to you, and what you can do with it: develop, validate, govern
3 Installing and Verifying the Runtime	How to prove you have the frozen build, and where your code goes
4 The Calls You Actually Need	The two dozen calls a first application uses, and what to ignore
5 Tutorial: A Battery-Powered Sensor	A complete application in five runnable steps, with tests
6 Tutorial, Continued: Stopping an Action That Loses Its Authority	A sixth step: a monitor withdraws a permit while the sensor recharges, and the runtime governs the stop
7 Seven Things the Runtime Will Not Do for You	The host duties that matter most, and the habit for each
8 Developing with AI	The traps, a five-step workflow, the rules, and three prompts
9 Where to Go Next	Which book answers which question

Chapter 2 — What the Runtime Does

The Productive Value–Productive Power (PV-PP) runtime answers one kind of question: given where a system stands now, which of the courses of action open to it keeps it inside the conditions for viable operation? Those conditions have two sides. The lower side is what the system must not fall below, such as enough energy, money, capability, or recovery capacity to keep going. The upper side is what it must not exceed, such as exposure, reach, resource use, or authority beyond what it is permitted or can safely hold (Section 1.3). The runtime answers the same way every time, by running a fixed sequence of stages, and it will not let a later stage cover for an earlier failure. If a stage cannot do its job, the decision stops there and the result says which stage stopped it and why (Reference 3.1).

This chapter is a summary. For a complete description of how the runtime works and why it is built this way, read the Programmer’s Guide: its Chapter 6 walks through the decision cycle stage by stage, its Chapter 7 covers selection and execution, and its Chapter 12 covers supervising an action while it runs. The exact contract of each stage is in the Programmer’s Reference (Reference Chapter 3), and the supervision layer’s in Reference Chapter 16.

2.1 Who Does What

Table 2.1 — The host and the runtime

	Supplies or does
Your code (the host)	The registry: what exists, meaning the domains and the levels at which they fail, the productive powers that carry them, the actions, the recovery routes, the hard rules, and the options available. The facts: the current state, and how each action changes it, through a world adapter. The consequences: what each option would do over the horizon, through a projection service. The action itself, carried out in the world, and the change to the recorded state, through a Layer-1 transition service. Optionally, monitoring tools that report facts while an action runs, and controllers that carry out the stop requests your code issues (Chapter 6).
The runtime	Forms the perceived state the decision is made on, runs the stages in order, and either selects one option or stops with a stated reason. When asked, it licenses the selected option for execution, checks each report of how execution went, and checks the change of state that results. With the supervision layer of build 167, it also keeps durable records of an action while it runs, decides at each checkpoint whether the action may continue, validates and records any stop request your code issues, and records what your controller reports.

2.2 The Stages

Each stage answers one question and uses the answers of the stages before it. It may not revise them.

Table 2.2 — The canonical order

Stage	The question it answers	Reference
Perception	What does the system believe about its own state? The decision is made on this, not directly on the host's record.	5.3
Φ and H	How strongly is each domain under strain, and how long until it fails if nothing new is done?	3.3
G and $\mathcal{R}$	Which domains must be preserved for recovery to stay possible, and how serious is the situation?	3.4
Graph and Π	Which options are structurally possible and reachable, and which candidates do they yield?	3.5
Π completeness	Is every kind of option that must be considered actually present?	3.6
Constraints	Which candidates are feasible under the rules? Most ceilings are enforced here, as hard rules.	3.7
Domain framing	What exactly is being preserved in each governing domain?	3.8
Adequacy	Which candidates keep the system viable until recovery is reachable and reach a region from which it can recover?	3.9; 3.10
Σ	Among the adequate candidates, which one? Ties that remain are broken by a fixed order you declare.	3.11
ϵ and Layer 1	Only when the host asks, after the cycle: the runtime licenses the selected option; the host carries out the action by its own means and reports what happened, which the runtime checks; the host then changes its recorded state through its Layer-1 transition service, and the runtime validates the result.	3.12

A decision cycle runs from perception through Σ and changes nothing. Acting on the decision is a separate step that you request (Chapter 5, step 5).

2.3 What Build 167 Adds

Build 167 leaves every stage of Table 2.2 exactly as it was in build 141 and adds a supervision layer around ϵ . In build 141, once an action was licensed, the runtime heard about it only when the host reported how it went. In build 167 the host can also register the action as active while it runs, feed in reports from monitoring tools, and checkpoint the action whenever a fact it depends on changes. At each checkpoint the runtime decides whether the action may continue. If it may not, your code fences the authority not yet used and issues a stop request; the runtime checks that the decision permits that request and records it, and an adapter translates it into your controller's own command.

Build 167 adds host-driven in-flight governance. Facts can change while an action is under way; when your instrumentation reports a change and your code calls the supervision layer, the

runtime admits the evidence and governs continuation at a checkpoint. The decision core remains synchronous. The runtime does not sense the world, schedule checkpoints, or stop processes on its own; your controllers do the stopping (Chapter 6; Reference Chapter 16).

2.4 What the Runtime Does Not Do

- **It does not know your domain.** Thresholds, rates, and projected consequences are claims you make. The runtime checks that they are complete, consistent, and well-formed. It cannot check that they are true.
- **It does not act on the world.** It decides and licenses. Your code acts, and reports what happened. When an action is supervised, the runtime may also decide that it must stop and validate the stop request your code then issues; your controller carries the stop out and reports the outcome.
- **It does not run by itself.** Every decision cycle and every checkpoint is started by the host. The runtime does not sense the world in the background, and it never interrupts a call or ends a process. Build 167 can decide, while an action is under way, that it must stop, but only when your code checkpoints the action, and the stop itself is requested by your code and carried out by your controller (Chapter 6; Reference 15.1, L1).
- **It keeps few records between runs.** The decision core's licenses, episodes, and ledgers live in memory in one runtime object and are not saved (Reference 15.1, L4). The supervision layer keeps its records in a durable store, but the live link between a stored execution and the runtime object is not saved. After a restart, an execution from before it can be reconciled and audited, but every operation that needs its live runtime state fails closed (Reference 16.9).

Facts in, one governed decision out, and every change to state made by the host through a checked boundary.

2.5 What You Can Do with the PV-PP Framework

Everything the PV-PP framework and its runtime offer falls under three activities, in the order an agent meets them: develop it, validate it before rollout, and govern it in production. Table 2.3 gives the promise of each and the four things it lets you do. Every item but one is available in runtime v2.1.167; the one marked Research has been demonstrated in controlled experiments and is not yet part of the released runtime.

Table 2.3 — Develop, validate, and govern

Develop	Validate	Govern
Design agents around what they must protect.	Prove the whole environment before rollout.	Control agents in production.
Define what the agent must keep viable, not just what it should maximize.	Run agents through realistic scenarios in a controlled, repeatable simulation.	Authorize every action at the moment it happens, never from stale permission.
Build that model with an AI assistant, guided by ready-made rules and prompts (Chapter 8).	Stress them with timing problems, failures, crashes, and conflicting signals.	Watch for changes while work runs, and revoke authority when the facts change.
Test each decision as you build, with a stated reason for every yes and no.	Check that every safeguard holds: stops reach the right place, and nothing slips past authorization.	Stop running work through your own systems, and confirm what actually happened.
Shape behavior through earned privileges: access lost for unauthorized moves, won back by acting within bounds. (Research)	Confirm the agent recovers, and that unknown outcomes stay flagged as unknown.	Keep a complete audit trail of every decision, stop, and outcome.

What the PV-PP framework does not claim: it is not an agent builder or a development environment; it works with your identity, security, and control systems rather than replacing them; and it does not stop work already in progress by itself—your systems carry out every stop it requests (Chapter 7).

For the build-by-build comparison—what the framework provides for, and what builds 131, 141, and 167 each implement—see Reference Appendix D.8.

Chapter 3 — Installing and Verifying the Runtime

The runtime is a Python package, `pvpp_runtime`, in the folder `runtime-v2.1.167` of the Productive Value–Productive Power (PV-PP) runtime repository, github.com/AmundsenLance/pvpp-runtime-api. This book uses runtime v2.1.167, the frozen release of build 167. Frozen means exactly what it says: you use the package, you never edit it, and you can prove that the copy you have is the frozen one. The release is also published as one archive, `PVPP_Runtime_v2.1.167_FROZEN.zip`. If you download the archive, check its SHA-256 before you unpack it: `7359f88ba83004af078e1f0cd26f1bb045bbb137c96e70044c3e0eb93087a85e`.

3.1 What You Need

- Python 3.11 or later. The package declares `requires-python ">=3.11"`.
- `pytest`, to run the frozen test suite and the tutorial's tests (`pip install pytest`).
- The `runtime-v2.1.167` folder from the repository, by cloning it, or the release archive. The package has no other dependencies; the supervision layer uses Python's built-in `sqlite3`.

3.2 Verify That You Have the Frozen Build

The folder contains a manifest, `V0167_FILE_HASHES.sha256`, listing the SHA-256 hash of every runtime source file, test, benchmark, and record: 346 manifest entries. If every hash matches, you have build 167. Then run the frozen test suite.

Listing 3.1 — Verifying the runtime

```
# From the runtime-v2.1.167 folder (the folder holding V0167_FILE_HASHES.sha256)
sha256sum -c V0167_FILE_HASHES.sha256 --quiet && echo "runtime v2.1.167 verified"
# macOS: shasum -a 256 -c V0167_FILE_HASHES.sha256 --quiet && echo "runtime v2.1.167 verified"

# The frozen regression suite; expected result: 1320 passed
PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider
```

If a hash fails, stop: whatever you have is not build 167, and nothing in these books can be relied on for it. Running the suite this way writes no cache files into the runtime folder. The folder also keeps `V0141_FILE_HASHES.sha256`, the manifest of build 141, as a record of where build 167 came from. Do not verify build 167 against it: one file, `pyproject.toml`, differs by design, because it carries the new package metadata.

3.3 Keep Your Code Outside the Runtime

Your application is host code. It lives in its own folder and imports the runtime from wherever it is installed. The tutorial's programs find it through one environment variable.

Listing 3.2 — Pointing your programs at the runtime

```
# Tell your programs where the runtime is (use your own path)
export PVPP_RUNTIME_PATH="$HOME/pvpp-runtime-api/runtime-v2.1.167"

# Your code lives in its own folder, never inside runtime-v2.1.167
mkdir battery && cd battery
```

Three rules follow from the runtime being frozen. Never edit a file in `runtime-v2.1.167`. Never patch runtime internals at run time or depend on names that begin with an underscore. And if

you need behavior the runtime does not have, build it in your own code; a changed runtime is not v2.1.167 (Reference 1.4).

Chapter 4 — The Calls You Actually Need

The runtime’s decision core exports 335 names, its core classes add 133 methods, and 11 more are reachable only through its submodules: 479 in all. The supervision layer of build 167 adds 38 names in `pvpp_runtime.supervision`. The Reference accounts for every one, which is why it is long. Only 78 of the core’s names are the ordinary supported API, and a first application needs about two dozen, most of them registration calls. Table 4.1 groups them by the tutorial step that uses them. Every runtime name in it is labeled PUBLIC / SUPPORTED in Appendix A of the Reference; the three service rows are protocols your code implements. The supervision calls of step 6 are listed separately, in Table 6.3.

Table 4.1 — The calls a first application needs

Purpose	Call	What it is for	Reference
Declare	<code>PVPPRegistry</code>	Holds everything the runtime may consider	4.2
	<code>register_domain</code>	A domain and the level at which it fails	4.3
	<code>register_power</code> ; <code>register_action</code>	The productive power that carries a domain; the actions, including “steady”	4.4
	<code>register_recovery_plan</code> ; <code>register_constraint_rule</code>	How a function recovers; a hard rule	4.5
	<code>register_governing_configuration</code> ; <code>register_regime_configuration</code> ; <code>register_sigma_order</code>	Which domains govern; the thresholds for how serious the situation is; the tie-break order	4.6
	<code>register_graph_instance</code> ; <code>register_graph_transformation</code>	Where the system stands; the options from there	4.7
Describe	World adapter	Perception, domain values, the effect of one step of an action, and rule checks	10.2
	<code>PolicyProjectionService</code> , returning <code>PolicyProjectionRecord</code>	What each candidate would do over the horizon	10.3; 9.6
	Layer-1 transition service	The only place recorded state changes	10.4
Decide	<code>PVPPRuntime</code>	Binds the registry to your adapter and services	4.1
	<code>CanonicalDecisionCycleRequest</code>	What the decision must preserve, and over what horizon	9.2
	<code>evaluate_integrated_canonical_cycle</code>	Perceives, runs every stage, and returns the result	9.3; 9.4

Purpose	Call	What it is for	Reference
Act	<code>build_execution_license_from_cycle</code>	Permission to carry out the selected option	11.2
	<code>instantiate_execution</code> ; <code>advance_execution</code> with <code>ExecutionObservation</code>	Open an episode; report how it went	11.3; 11.4
	<code>build_layer1_transition_handoff</code> ; <code>apply_layer1_transition</code>	Hand the outcome to your transition service	11.10
	<code>build_execution_transition_provenance</code>	Tie the new state to the decision that caused it	11.11

Each registration call takes a definition object, such as `DomainDefinition` or `ActionDefinition`, documented in the same Reference section as the call. Table 5.2, at the end of the tutorial, maps these calls to the code that uses them.

4.1 What to Ignore at First

Leave everything labeled PUBLIC / ADVANCED until you have a reason to use it; the Reference says, for each one, when that is. Leave IMPLEMENTATION / SUPPORT and INTERNAL entries alone: they are how the runtime does its work, not how you use it. And never use anything labeled LEGACY / COMPATIBILITY (Reference 15.4).

The legacy entries remain in the runtime so that older code keeps working, and the runtime deliberately keeps tests that prove they still do. Most of the example scripts shipped with the runtime use legacy entries for the same reason: they come from earlier runtime generations, and many are the small worlds those compatibility tests run on. They are not tutorials that were left behind, and they are not patterns for new code. Learn from this book and the Guide. Of the shipped examples, read `examples/authority_bound_callthrough_v0130.py`; `examples/minimal_battery_service` is kept for compatibility and carries a known sign error.

The sign error in `minimal_battery_service`. Its `expected_deterioration` returns a positive magnitude, and its comment says that is what the runtime expects. It is not: the runtime reads a positive value as improvement, so that example's time-to-failure horizons come out effectively infinite, and only its pressure values reach the regime. Its decision is still the right one, because selection turns on projections and adequacy. The example is part of the frozen runtime and stays as it is; follow step 2 of the tutorial, which returns the signed change (Section 5.2).

The supervision layer has guards of its own. Once an execution is registered for supervision, the bridge's `invoke_authorized_native` and its direct `advance_execution` are refused; they remain only for executions that never entered supervision, and are labeled LEGACY / COMPATIBILITY guards. The supervisory store's tables and private helpers are INTERNAL: use the services (Reference 16.1).

4.2 When a Call Refuses

The runtime separates two kinds of failure. When the facts rule something out, such as no feasible candidate or no adequate one, that is an outcome: the call returns normally, with a status in its result. When your code misuses a call or breaks a host contract, such as a missing hook or a license used twice, that is an error, and the call raises an exception (Reference 14.1). So read the status before using anything else in a result. Reference 14.11 is a step-by-step guide

to finding which stage refused and why, and Appendix C of the Reference indexes every status. The supervision layer follows the same rule and adds typed errors your code should handle one by one: `ConcurrencyConflict` means another operation got there first and a retry may succeed; `ContinuationReconciliationRequired` means a retry will not, and the execution needs authorized reconciliation. Test for the second first: it is a kind of `ConcurrencyConflict` (Reference 16.11).

Chapter 5 — Tutorial: A Battery-Powered Sensor

This chapter builds a complete Productive Value–Productive Power (PV-PP) application in five steps. Each step is one small file that runs on its own and prints what it has built, so you can stop after any step and see where you are.

The application is a sensor that runs on a battery. It can take measurements, which drain the battery, or recharge. What must be preserved is its ability to keep measuring: if usable energy falls to 2.0 units, the sensor fails. Each period uses 0.5 units, whether it samples or idles; a recharge restores the battery to 8.0. The decision the runtime makes is simple to state and worth getting right: keep sampling, or recharge now?

Put the six files of this chapter in one folder (exact copies are in the repository's getting-started/battery folder), set PVPP_RUNTIME_PATH as in Section 3.3, and run each step with python3.

Table 5.1 — The five steps

Step	File	What it adds
1	step1_registry.py	What exists: the domain, actions, recovery route, rules, and options
2	step2_world.py	The world adapter: what the sensor perceives and how the world moves
3	step3_projection.py	The projection service: what each option would do
4	step4_decide.py	One decision cycle, and how to read its result
5	step5_act.py	Carrying out the decision and changing the recorded state

5.1 Step 1: Declare What Exists

The registry is the list of everything the runtime may govern. Nothing the runtime considers can come from anywhere else. For the sensor it holds one domain (energy) with its failure threshold, the productive power that carries it (battery capacity), three actions, a recovery plan saying that recharging is how the measurement function recovers, the thresholds that decide how serious the situation is, one hard rule, and the graph—the options the sensor has from where it stands.

Listing 5.1 — step1_registry.py

```

"""Step 1: declare what exists. A battery-powered sensor with one domain and three actions."""

import os, sys

# Point PVPP_RUNTIME_PATH at the runtime folder (the folder that contains pvpp_runtime/).
if os.environ.get("PVPP_RUNTIME_PATH"):
    sys.path.insert(0, os.environ["PVPP_RUNTIME_PATH"])

from pvpp_runtime import (
    PVPPRegistry, DomainDefinition, ProductivePowerDefinition, ActionDefinition,
    RecoveryPlanDefinition, GoverningConfiguration, RegimeConfiguration,
    ConstraintRuleDefinition, GraphInstanceDefinition, GraphTransformationDefinition,
    SigmaOrderDefinition,
)

DOMAIN = "energy"                                # what must stay viable

```

```

POWER = "battery_capacity"      # the productive power that carries it
FUNCTION = "measurement"        # what the energy is for

def build_registry(order=("graph:continue_sampling", "graph:restore_battery")):
    r = PVPPRegistry()
    # The domain fails if usable energy falls to 2.0 units.
    r.register_domain(DomainDefinition(DOMAIN, "usable battery energy", threshold=2.0))
    r.register_power(ProductivePowerDefinition(POWER, DOMAIN, "capacity to power sensing"))

    # Every registry needs "steady": what happens if the sensor does nothing new.
    for action, meaning in (("steady", "idle drain"),
                            ("sample", "take a measurement"),
                            ("recharge", "restore the battery")):
        r.register_action(ActionDefinition(action, meaning, (DOMAIN,)))

    # Recharging is the recovery route for the measurement function.
    r.register_recovery_plan(RecoveryPlanDefinition(
        "battery_recovery", DOMAIN, FUNCTION, "recharge", (), deadline_offset=2.0))

    # How close to failure counts as Stabilization, Survival, or Existential.
    r.register_governing_configuration(GoverningConfiguration(epsilon_h=0.0))
    r.register_regime_configuration(RegimeConfiguration(
        tau_h_existential=1.0, tau_h_survival=3.0, tau_h_stabilization=8.0,
        tau_phi_existential=10.0, tau_phi_survival=2.0, tau_phi_stabilization=0.5))

    # One hard rule. The host reports whether a candidate breaks it.
    r.register_constraint_rule(ConstraintRuleDefinition("action_allowed", "hard"))

    # The graph: two ways the sensor can go on from here.
    r.register_graph_instance(
        GraphInstanceDefinition("sensor", "productive_system", (DOMAIN,), "active"))
    r.register_graph_transformation(GraphTransformationDefinition(
        "continue_sampling", "sensor", "sensor", "continuation", (DOMAIN,),
        "reachable", "sample", ("continuation",)))
    r.register_graph_transformation(GraphTransformationDefinition(
        "restore_battery", "sensor", "sensor", "corrective_repair", (DOMAIN,),
        "reachable", "recharge", ("recovery",), structurally_required=True))

    # Tie-break order, used only when two options are equally good. Lower wins.
    for rank, policy in enumerate(order, start=1):
        r.register_sigma_order(SigmaOrderDefinition(policy, 10 * rank))
    return r

if __name__ == "__main__":
    r = build_registry()
    print("domains:      ", tuple(r.domains))
    print("actions:      ", tuple(r.actions))
    print("transformations:", tuple(r.graph_transformations))

```

Three lines deserve a second look. The action `steady` is required: it is what happens if the sensor does nothing new, and the runtime measures pressure and time-to-failure against it. The recovery transformation is marked `structurally_required`, so recharging is always among the options the runtime must consider. And the Σ order at the end is not a preference. It is used only to break a tie between options that are otherwise exactly as good; step 4 shows one.

Listing 5.2 — Output of step 1

```

domains:      ('energy',)
actions:      ('steady', 'sample', 'recharge')
transformations: ('continue_sampling', 'restore_battery')

```

5.2 Step 2: Tell the Runtime What the World Looks Like

The world adapter answers the questions the runtime asks about your domain. It never decides anything. Its most important method forms the perceived decision state—what the sensor believes about itself—from the host’s record of the actual sensor. Here perception is exact; in a real application it is where noisy or stale evidence is weighed (Guide 8.5). The other methods say how one step of an action changes the battery, how far the domain is from failure, and whether a candidate breaks the hard rule; in this example nothing is forbidden.

One sign matters. `expected_deterioration` returns the signed change per period when the sensor does nothing new: here `-0.5`, because the battery is falling. The runtime divides the margin above the threshold by that rate to get the time to failure, so a positive number would tell it the battery is improving, and every horizon would come out effectively infinite. Nothing raises an error if you get this wrong; the regime in step 4 is where it shows.

Listing 5.3 — `step2_world.py`

```
"""Step 2: tell the runtime what the world looks like. The world adapter states facts."""

from step1_registry import DOMAIN, POWER
from pvpp_runtime import (
    WorldState, ProductivePowerState, PerceivedDecisionState, ActualPersistentStateEnvelope,
    ActionProjection, PressureFactors, PolicyConstraintProfile, ConstraintObservation,
)

DRAIN = 0.5          # energy used per period, idle or sampling
FULL = 8.0           # battery level after a recharge

def actual_state(battery):
    """The host's record of the real sensor: its actual persistent state."""
    return ActualPersistentStateEnvelope(
        "sensor-A", f"battery-{battery}", 0.0,
        {"battery_units": battery}, {"stored_charge": battery},
        {"service_available": True}, {})

class World:
    """The world adapter. It answers the runtime's questions; it never decides."""

    # Perception: what the sensor believes about itself, formed from its actual state.
    def perceived_decision_state_from_actual(self, actual):
        b = float(actual.pp["battery_units"])
        seen = WorldState(actual.time, {POWER: ProductivePowerState(POWER, b)},
                           {"state_id": actual.state_id})
        return PerceivedDecisionState(actual.actor_id, actual.state_id, actual.time,
                                       seen, ppp={POWER: b})

    def perceive(self, state):          # already perceived: return it unchanged
        return state

    def domain_value(self, state, domain_id):
        return state.powers[POWER].value

    # One step of an action, used for the baseline ("steady") and horizons.
    def project(self, state, action):
        b = self.domain_value(state, DOMAIN)
        b = FULL if action.id == "recharge" else b - DRAIN
        return ActionProjection(action.id, WorldState(
            state.time + 1, {POWER: ProductivePowerState(POWER, b)}, dict(state.metadata)), True)

    # Pressure: how far above failure the domain is, and how fast it is falling.
    def pressure_factors(self, state, domain_id, baseline_drift):
        return PressureFactors(domain_id, self.domain_value(state, domain_id) - 2.0,
                               baseline_drift)
```

```

def pressure_value(self, domain_id, factors):
    return 1.0 / max(factors.margin, 0.1)

def expected_deterioration(self, state, domain_id, baseline_drift, pressure_value,
                           pressure_factors):
    # Signed change per period under "steady": negative means the domain is falling.
    return float(baseline_drift)

# Constraints: nothing is forbidden in this example.
def constraint_profile(self, state, policy_id, action_ids, regime, governing_domain_ids):
    return PolicyConstraintProfile(policy_id,
                                   (ConstraintObservation("action_allowed", False),))

if __name__ == "__main__":
    p = World().perceived_decision_state_from_actual(actual_state(9.0))
    print("state:", p.state_id, "| perceived battery:", p.represented_state.powers[POWER].value)

```

Listing 5.4 — Output of step 2

```
state: battery-9.0 | perceived battery: 9.0
```

Notice the two states. The actual state is the host's record of the real sensor, and only the host changes it. The perceived decision state is what the decision is made on. Keeping them apart is how the PV-PP framework stays honest when the two disagree.

5.3 Step 3: Say What Each Option Would Do

To choose between sampling and recharging, the runtime needs to know what each would do to the battery over the next ten periods. That knowledge comes from a projection service—a separate object you attach to the runtime. For each candidate it returns one projection record: when the domain would fail, or that it would still be safe at the end of the horizon, and whether the sensor would stay in, or reach, a region from which it can recover.

Listing 5.5 — step3_projection.py

```

"""Step 3: say what each option would do. The projection service projects; it never chooses."""

from step1_registry import DOMAIN, POWER, FUNCTION
from step2_world import DRAIN, FULL, World, actual_state
from pvpp_runtime import (
    WorldState, ProductivePowerState, PolicyProjectionRecord, RecoveryCorridorProjection,
    ProjectionRequest,
)

class Projection:
    """A PolicyProjectionService: one projection record per candidate policy."""

    def project(self, req):
        start = req.represented_state.powers[POWER].value
        horizon = req.projection_horizon
        if "recharge" in req.action_ids:
            end, fails_at = FULL, None  # recharged: safe past the horizon
        else:
            end = start - DRAIN * horizon  # sampling all the way to the horizon
            t = (start - 2.0) / DRAIN  # periods until the 2.0 threshold
            fails_at = t if t < horizon else None
        safe = fails_at is None
        corridor = RecoveryCorridorProjection(
            DOMAIN, FUNCTION,
            pre_recovery_viability_preserved=safe,
            recovery_capable_region_reached=safe,  # recharging reaches it; a healthy

```

```

    joint_sustainment_supported=True,          # sensor never leaves it
    recovery_entry_time=(1.0 if "recharge" in req.action_ids else 0.0) if safe else None,
    projected_extinction_time=fails_at)
return PolicyProjectionRecord(
    req.policy_id, False,
    WorldState(req.represented_state.time + 1,
               {POWER: ProductivePowerState(POWER, max(end, 0.0))},
               dict(req.represented_state.metadata)),
    {DOMAIN: fails_at if fails_at is not None else horizon},
    (corridor,)),
    # Echo the request so the runtime can check that this record answers it.
    projection_horizon=horizon,
    right_censored_domain_ids=(DOMAIN,) if safe else (), # "still fine at the horizon"
    state_id=req.state_id, state_time=req.represented_state.time,
    candidate_mode=req.candidate_mode, model_version="battery-q1",
    projection_input_trace=dict(req.projection_input_trace))

if __name__ == "__main__":
    for battery in (9.0, 5.0):
        seen = World().perceived_decision_state_from_actual(actual_state(battery))
        for policy, action in (("graph:continue_sampling", "sample"),
                               ("graph:restore_battery", "recharge")):
            q = Projection().project(ProjectionRequest(
                seen.represented_state, policy, (action,), "ordinary_adequacy", 10.0,
                state_id=seen.state_id))
            h = q.projected_horizons[DOMAIN]
            note = "safe through the horizon" if q.right_censored_domain_ids else "fails at"
            print(f"battery {battery}: {policy:24} {note} {h}")

```

The record echoes the request—its state, horizon, and candidate—so the runtime can check that the record answers the question it asked. A domain still safe at the horizon is marked right-censored: the service says “fine for as long as I looked”, not an invented time of failure. Run it, and the arithmetic is visible: from 9.0 units, sampling stays above 2.0 for all ten periods; from 5.0 it reaches the threshold after six.

Listing 5.6 — Output of step 3

```

battery 9.0: graph:continue_sampling  safe through the horizon 10.0
battery 9.0: graph:restore_battery    safe through the horizon 10.0
battery 5.0: graph:continue_sampling  fails at 6.0
battery 5.0: graph:restore_battery    safe through the horizon 10.0

```

Use a projection service, as here, rather than the older projection method on the world adapter. The runtime licenses and validates everything a service returns; the older route receives none of those checks (Reference 10.3).

5.4 Step 4: Run One Decision Cycle

Now the runtime can decide. The request says what the decision must preserve (the sensor’s ability to measure), which domain is at stake, which kinds of option must be present, and how far ahead to project. `evaluate_integrated_canonical_cycle` takes the actual state, perceives it through the adapter, and runs every stage in order. Nothing is executed: the result is a decision, not an action.

Listing 5.7 — `step4_decide.py`

```

"""Step 4: run one decision cycle and read what came back. Nothing is executed yet."""

from step1_registry import DOMAIN, FUNCTION, build_registry
from step2_world import World, actual_state
from step3_projection import Projection
from pvpp_runtime import (
    PVPPRuntime, CanonicalDecisionCycleRequest, PreliminaryPreservationObject,
    DomainFrame, DomainFrameTarget,

```

```

)

def make_runtime(layer1=None):
    return PVPPRuntime(build_registry(), World(), projection_service=Projection(),
                       layer1_transition_service=layer1)

def request():
    return CanonicalDecisionCycleRequest(
        PreliminaryPreservationObject("keep_measuring", "keep the sensor able to measure"),
        DomainFrame((DomainFrameTarget(DOMAIN, FUNCTION),)),
        required_graph_family_ids=("continuation", "corrective_repair"),
        materially_required_policy_class_ids=("continuation", "recovery"),
        projection_horizon=10.0)

def decide(battery, rt=None):
    rt = rt or make_runtime()
    return rt, rt.evaluate_integrated_canonical_cycle(actual_state(battery), request())

if __name__ == "__main__":
    for battery in (9.0, 5.0):
        _, out = decide(battery)
        d = out.decision
        print(f"\nbattery {battery}: {out.status}")
        print("  stages:  ", " " -> ".join(d.pipeline_trace))
        print("  regime:   ", d.regime_assessment.regime)
        print("  candidates:", tuple(c.id for c in d.pi_construction.policy_space.candidates))
        print("  adequate:  ", d.adequacy.adequate_policy_ids)
        print("  finalists: ", d.selection.sigma.final_policy_ids)
        print("  selected:  ", d.selection.selected_policy_id)

```

Listing 5.8 — Output of step 4

```

battery 9.0: integrated_cycle_selected_not_executed
  stages:  PPP -> Phi -> H -> G -> R -> Graph/Seed -> Pi -> Pi Completeness -> Constraints -> Domain
    Framing -> Adequacy -> Sigma
  regime:  Mission
  candidates: ('graph:continue_sampling', 'graph:restore_battery')
  adequate:  ('graph:continue_sampling', 'graph:restore_battery')
  finalists: ('graph:continue_sampling', 'graph:restore_battery')
  selected:  graph:continue_sampling

battery 5.0: integrated_cycle_selected_not_executed
  stages:  PPP -> Phi -> H -> G -> R -> Graph/Seed -> Pi -> Pi Completeness -> Constraints -> Domain
    Framing -> Adequacy -> Sigma
  regime:  Stabilization
  candidates: ('graph:continue_sampling', 'graph:restore_battery')
  adequate:  ('graph:restore_battery',)
  finalists: ('graph:restore_battery',)
  selected:  graph:restore_battery

```

Read the two cases stage by stage. With 9.0 units both options keep the sensor safe through the horizon, so both are adequate and both reach the end of Σ as finalists; the tie is broken by the declared order, and the sensor keeps sampling. With 5.0 units sampling would fail in six periods. Adequacy removes it, and recharging is the only option left. No code in the application says “recharge when the battery is low”: the rule falls out of the facts, the threshold, and the stages.

The regime line shows how serious the runtime judges each situation. With 9.0 units and nothing new done, the sensor has fourteen periods before it fails, more than the eight that step 1 declared as the edge of Stabilization, so the regime is Mission. With 5.0 units it has six, so the

regime is Stabilization. Regime is a posture, not a verdict: it shapes which kinds of option are generated, and it never decides adequacy or selection (Reference 3.4).

Change the numbers and run it again. Try 6.5, or change DRAIN in step 2, and predict the answer before you look.

5.5 Step 5: Carry Out the Decision

A decision is permission, not an event. To act, the host asks for a license built from the decision, opens an execution episode, performs the action by its own means, reports what happened, and then applies the change to its own record through a Layer-1 transition service. The runtime checks each step; the host does the work.

Listing 5.9 — step5_act.py

```
"""Step 5: carry out the decision and apply the state change. The host owns the change."""

from step2_world import FULL, DRAIN
from step2_world import actual_state
from step4_decide import decide, make_runtime, request
from pvpp_runtime import (
    ActualPersistentStateEnvelope, Layer1TransitionResult, ExecutionObservation,
)

class Layer1Service:
    """The host's only way to change the sensor's recorded state."""

    def transition(self, current, handoff):
        pp = dict(current.pp)
        if "recharge" in handoff.licensed_action_ids:
            pp["battery_units"] = FULL
        else:
            pp["battery_units"] = float(pp["battery_units"]) - DRAIN
        nxt = ActualPersistentStateEnvelope(
            current.actor_id, current.state_id + ":next", current.time + 1.0,
            pp, current.spv, current.avs, dict(current.context))
        return Layer1TransitionResult(
            handoff.episode_id, handoff.selected_policy_id, current.state_id, nxt, True,
            {"battery_nonnegative": pp["battery_units"] >= 0.0})

def act(battery):
    rt, out = decide(battery, make_runtime(Layer1Service()))
    actual = actual_state(battery)
    # 1. A license: the runtime's permission to carry out this one decision.
    license = rt.build_execution_license_from_cycle(out.decision, request().domain_frame)
    # 2. An execution episode. The host asserts the action can start now.
    step = rt.instantiate_execution("episode-1", license, entry_sufficient=True, max_steps=1)
    # 3. The host performs the action by its own means, then reports what happened.
    step = rt.advance_execution(step.episode, ExecutionObservation("obs-1", completed=True))
    # 4. The host applies the state change through its Layer-1 service; the runtime checks it.
    handoff = rt.build_layer1_transition_handoff(step)
    result = rt.apply_layer1_transition(actual, handoff)
    check = rt.validate_layer1_transition_result(actual, handoff, result)
    provenance = rt.build_execution_transition_provenance(actual, handoff, result, check)
    return license, step, result, provenance

if __name__ == "__main__":
    license, step, result, provenance = act(5.0)
    print("licensed actions:", license.action_ids)
    print("execution status:", step.status)
    print("battery:      5.0 ->", result.next_state.pp["battery_units"])
    print("new state id:   ", provenance.next_state_id)
```

Listing 5.10 — Output of step 5

```

licensed actions: ('recharge',)
execution status: completed
battery:         5.0 -> 8.0
new state id:    battery-5.0:next

```

In this application the sensor’s recorded state changes only through the transition service, as the host contract requires; the runtime cannot stop host code from changing it elsewhere (Chapter 7). The service gives the new state a new identity. The provenance record ties that new state to the license, the episode, and the decision that caused it. Two rules that the runtime cannot enforce for you: carry out each decision at most once, and treat the first terminal result, whether completed or failed, as final (Reference 11.2–11.4).

5.6 What You Just Used**Table 5.2 — The calls in this tutorial and where the Reference documents them**

Purpose	Calls	Reference
Declare what exists	PVPPRegistry; register_domain, register_power, register_action, register_recovery_plan, register_governing_configuration, register_regime_configuration, register_constraint_rule, register_graph_instance, register_graph_transformation, register_sigma_order	4.2–4.7
Describe the world	The world adapter’s hooks; a PolicyProjectionService	10.2; 10.3
Decide	PVPPRuntime; CanonicalDecisionCycleRequest; evaluate_integrated_canonical_cycle	4.1; 9.2–9.4
Act	build_execution_license_from_cycle; instantiate_execution; advance_execution with an ExecutionObservation; build_layer1_transition_handoff; apply_layer1_transition; validate_layer1_transition_result; build_execution_transition_provenance	11.2–11.4; 11.10–11.11
Change state	A Layer-1 transition service	10.4

All of these are PUBLIC / SUPPORTED or host protocols the runtime requires, with two exceptions worth knowing. `validate_layer1_transition_result` is PUBLIC / ADVANCED: `apply_layer1_transition` already validates, and step 5 calls it again only because the provenance record needs its result. And the demonstration at the end of step 3 builds a `ProjectionRequest` by hand; in an application only the runtime builds one. Nothing in the tutorial uses an internal or legacy entry.

5.7 Test It

A PV-PP application should be tested like any other program, with one addition: test the decisions, not just the code. These six tests check that the healthy sensor samples, the weak one recharges, the projection marks censoring honestly, acting recharges the battery, and—the instructive one—that reversing the tie-break order changes the choice only where there was a tie.

Listing 5.11 — test_battery.py

```

"""Tests for the tutorial. Run: PVPP_RUNTIME_PATH=<runtime folder> python3 -m pytest -q"""

from step1_registry import DOMAIN, build_registry
from step2_world import World, actual_state
from step3_projection import Projection
from step4_decide import decide, request
from step5_act import act
from pvpp_runtime import PVPPRuntime, ProjectionRequest

def test_registry_has_the_required_baseline_action():
    assert "steady" in build_registry().actions

def test_projection_reports_a_failure_without_censoring():
    seen = World().perceived_decision_state_from_actual(actual_state(5.0))
    q = Projection().project(ProjectionRequest(
        seen.represented_state, "graph:continue_sampling", ("sample",),
        "ordinary_adequacy", 10.0, state_id=seen.state_id))
    assert q.projected_horizons[DOMAIN] == 6.0 and q.right_censored_domain_ids == ()

def test_healthy_sensor_keeps_sampling():
    _, out = decide(9.0)
    assert out.status == "integrated_cycle_selected_not_executed"
    assert out.decision.horizon_assessment.domain_results[0].horizon == 14.0 # (9.0 - 2.0) / 0.5
    assert out.decision.regime_assessment.regime == "Mission"
    assert out.decision.selection.selected_policy_id == "graph:continue_sampling"

def test_low_battery_recharges_because_sampling_is_inadequate():
    _, out = decide(5.0)
    assert out.decision.regime_assessment.regime == "Stabilization" # 6 periods left, within 8
    assert out.decision.adequacy.adequate_policy_ids == ("graph:restore_battery",)
    assert out.decision.selection.selected_policy_id == "graph:restore_battery"

def test_order_only_breaks_ties():
    reversed_order = build_registry(order=("graph:restore_battery", "graph:continue_sampling"))
    rt = PVPPRuntime(reversed_order, World(), projection_service=Projection())
    healthy = rt.evaluate_integrated_canonical_cycle(actual_state(9.0), request())
    low = rt.evaluate_integrated_canonical_cycle(actual_state(5.0), request())
    assert healthy.decision.selection.selected_policy_id == "graph:restore_battery" # tie
    assert low.decision.selection.selected_policy_id == "graph:restore_battery" # not a tie

def test_acting_recharges_the_battery_with_valid_provenance():
    license, step, result, provenance = act(5.0)
    assert license.action_ids == ("recharge",) and step.status == "completed"
    assert result.next_state.pp["battery_units"] == 8.0
    assert provenance.next_state_id == result.next_state.state_id

```

Listing 5.12 — Running the tests

```

$ PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider
.....
6 passed

```

[100%]

5.8 Where to Go Next

The sensor has one domain, perfect perception, and nothing that can go wrong while it acts. Real applications have several domains that pull against each other, evidence that goes stale, rules that forbid, actions that fail halfway, and facts that change while an action is under way. Chapter 6 adds the last of these to the sensor. The Guide builds the rest one at a time on a

coffee-vending machine (Guide Chapters 6–9); Appendix B of the Reference has the same five pieces as this tutorial in reference form, plus re-entry after a change and a supervised execution.

Chapter 6 — Tutorial, Continued: Stopping an Action That Loses Its Authority

In Chapter 5 the recharge happened in one step: the host reported that it was complete, and nothing could change while it ran. Real actions take time, and the facts they depend on can change while they run. A permission is withdrawn, a resource disappears, a monitoring tool reports that a limit has been crossed. This chapter adds a sixth step to the tutorial. The recharge now takes time, a monitoring tool watches whether charging is permitted, and when the permit is withdrawn partway through, the runtime decides that the recharge may not continue, and the host asks the charger, through the runtime's governed control path, to stop it.

Everything in this chapter uses the supervision layer that build 167 adds, the package `pvpn_runtime.supervision`. Nothing in Chapter 5 changes. The decision cycle, the license, and the episode are the same objects, and the supervision layer works through them (Guide Chapter 12; Reference Chapter 16).

6.1 What Supervision Adds

The runtime's decision core is synchronous. It runs when your code calls it and returns when it is done, and it watches nothing. Build 167 keeps that core byte for byte and wraps it in a supervision layer that keeps durable records of an action while it runs, so that a change reported partway through can be governed rather than missed. The layer adds one chain of steps. Each link is a separate record, and each has a separate owner.

Table 6.1 — The supervision chain

Link	Who supplies it	What the runtime does
1. Report	Your monitoring tool, registered as a source	Records the report as evidence, never as a decision
2. Admission	The runtime	Checks that the source is registered, current, and in scope. Only an admitted report becomes a governed fact change
3. Fan-out	The runtime, when your code asks	Finds every running execution that declared a dependency on the changed fact
4. Checkpoint	Your code, for each affected execution	Decides whether the execution may continue; if not, records why and whether its unused authority must be fenced
5. Stop request	Your code, as the execution's current supervisory owner	Permits only the controls the checkpoint's decision allows, and records the request
6. Dispatch	Your adapter	Translates the request into your controller's own command, and refuses any translation that widens it
7. Enforcement	Your controller carries out the command and reports	Records the outcome, and whether the report's attestation verified

Three things follow from the table. They are the most important facts in this chapter.

- **The runtime decides; your controller stops.** The runtime never ends a process or interrupts a call. It records a governed decision that the action may not continue, and it validates and records the stop request your code issues. Your controller does the stopping and reports back, and its report may say accepted, completed, rejected, too_late, timed_out, or unknown. The runtime keeps each of those distinct.
- **A monitor reports; it never decides.** A report from an unregistered or out-of-scope source is retained as evidence and changes nothing. Even an admitted report only leads to a checkpoint, and the checkpoint decides.
- **Your code drives every step.** The runtime finds the affected executions, but it does not checkpoint them by itself. There is no background thread and no callback. If your code does not call the checkpoint, nothing is governed.

6.2 The Situation

The sensor has decided to recharge: step 4 with 5.0 units. Charging is done by an external charger, which takes several periods and runs the charge as a job it controls. Charging is allowed only while the site holds a charging permit, and a site monitor reports whether it does. The permit is a ceiling in the sense of Section 1.3: a limit on what the sensor is authorized to do, not on what it is able to do. Ten minutes into the charge, the monitor reports that the permit has been withdrawn.

Two parties in the example stand outside PV-PP and are written as small stand-ins in the file: the charger, which runs jobs and carries out commands, and the trust configuration, which says who may register what and how the charger's reports are verified. In a deployment both are yours: real controllers, and your own identity and key systems. The literal credentials in the file are for demonstration only.

6.3 Step 6: Supervise the Recharge

step6_supervise.py builds on steps 4 and 5. It has three functions, one for each phase, so that the tests can drive them separately: start_supervised_charge sets everything up and starts the charge, withdraw_permit reports the withdrawal and checkpoints, and stop carries out the stop.

Listing 6.1 — step6_supervise.py

```

"""Step 6: supervise the recharge while it runs. A site monitor withdraws the charging permit."""

from datetime import datetime, timezone
from types import SimpleNamespace

from step4_decide import decide, make_runtime, request
from step5_act import Layer1Service
from pvpp_runtime import ExecutionBindingRegistry, ExecutionBindingIdentity
from pvpp_runtime import GovernanceInvalidationSignal
from pvpp_runtime.supervision import (
    HostTrustProvider, SQLiteSupervisoryStore, CanonicalRuntimeBridge,
    ObservationService, SourceRegistration, ExternalObservation,
    DependencyService, ContinuationService, ControlService,
)

CFG = "battery-site"          # the configuration the supervised execution belongs to
SENSOR = "sensor-A"          # the subject the monitor reports on
ADMIN, PROOF = "site-admin", "site-admin-proof"  # demonstration credentials only

def at(minute):
    """Timestamps for the example: 12:00 UTC plus the given minute."""
    return f"2026-10-07T12:{minute:02d}:00+00:00"

def trust():
    """Who may register what, and how a controller's report is verified. In a deployment
    these come from your own identity and key systems, not from literals in code."""
    def charger_report_verified(**report):
        return report["attestation_evidence"] == "signed:charger"

    return HostTrustProvider(
        {ADMIN: {"proof": PROOF, "roles": ("*",), "configurations": (CFG,)}},
        controller_attestation_verifier=charger_report_verified,
        clock=lambda: datetime(2026, 10, 7, 12, 30, tzinfo=timezone.utc))

class Charger:
    """An external controller that runs charging jobs. It is not part of PV-PP."""

    def __init__(self):
        self.jobs = {}

    def start(self):
        # returns at once; the job keeps running
        handle = f"charger:job-{len(self.jobs) + 1}"
        self.jobs[handle] = "charging"
        return handle

    def send(self, command, handle):
        # carries out a vendor command
        if self.jobs.get(handle) != "charging":
            return "too_late", "signed:charger"
        if command == "stop_charge":
            self.jobs[handle] = "stopped"
        return "completed", "signed:charger"

def start_supervised_charge():
    """Decide, license, register the execution for supervision, and start the charge."""
    s = SimpleNamespace(charger=Charger())
    s.rt, out = decide(5.0, make_runtime(Layer1Service()))
    license = s.rt.build_execution_license_from_cycle(out.decision, request().domain_frame)

    # 1. Open the episode through the bridge and issue the authority for this execution.
    s.bridge = CanonicalRuntimeBridge(s.rt)
    episode = s.bridge.instantiate_execution("episode-1", license, entry_sufficient=True,
                                           max_steps=5).episode
    bindings = ExecutionBindingRegistry(tuple(s.rt.registry.actions))

```

```

bindings.register(ExecutionBindingIdentity("charge", "recharge", "1"), lambda ctx: None)
s.auth = s.bridge.issue_native_execution_authorization(
    episode, "recharge", bindings, decision_cycle_id="cycle-1", configuration_id=CFG)

# 2. Register the execution as active in a durable supervisory store.
s.store = SQLiteSupervisoryStore(":memory:", trust_provider=trust())
handle = s.bridge.resolve_canonical_execution(s.auth, configuration_id=CFG)
s.active = s.bridge.register_active_execution(
    s.store, handle, registered_at=at(0),
    execution_id=s.rt.native_execution_authorization_execution_id(s.auth.authorization_id))
aid = s.active.active_execution_id

# 3. Register the monitor as a source, admit its first report, declare the dependency.
s.obs, s.deps = ObservationService(s.store), DependencyService(s.store)
s.cont = ContinuationService(s.store, s.deps, s.bridge)
s.obs.register_source(SourceRegistration(
    "reg-monitor-1", "site-monitor", ("observation",), ("charge_permit",), (SENSOR,),
    (CFG,), ("governance",), "site-signature", None, None, "current", ADMIN, 1),
    authority_proof=PROOF)
report(s, "permit-1", "granted", at(0))
s.deps.register_dependency_binding(
    active_execution_id=aid, configuration_id=CFG, subject_id=SENSOR,
    fact_kind="charge_permit", fact_id="site", registration_authority=ADMIN,
    authority_proof=PROOF, registered_at=at(0))

# 4. Take supervisory ownership, and register the charger's control adapter.
sctl = ControlService(s.store, s.deps, s.cont, s.bridge)
sctl.register_supervisory_owner(owner_id="host-1", configuration_id=CFG,
    registration_authority=ADMIN, authority_proof=PROOF,
    registered_at=at(0))
s.owner = sctl.acquire_ownership(active_execution_id=aid, owner_id="host-1",
    acquired_at=at(0))
s.adapter = sctl.register_adapter(
    adapter_id="charger", configuration_id=CFG, target_handle_namespace="charger:",
    supported_semantic_controls=("pause", "cancel"),
    command_mapping={"pause": "pause", "cancel": "cancel"},
    vendor_command_mapping={"pause": "hold_charge", "cancel": "stop_charge"},
    controller_attestation_method="signed", control_region_evidence_method="charger-state",
    registration_authority=ADMIN, authority_proof=PROOF, registered_at=at(0))

# 5. The host starts the charge by its own means. The episode stays open while it runs.
s.job = s.charger.start()
return s

def report(s, observation_id, value, when, source="site-monitor"):
    """One monitor report: submit it, assess its admission, commit the governed change."""
    o = ExternalObservation(observation_id, source, "charge_permit", "site", SENSOR, CFG,
        value, None, when, when, when, {})
    s.obs.submit_observation(o)
    return s.obs.commit_admission_change(o, s.obs.assess_observation(o, assessed_at=when),
        committed_at=when)

def withdraw_permit(s):
    """6-7. The monitor withdraws the permit; the host checkpoints every affected execution."""
    change = report(s, "permit-2", "withdrawn", at(10))
    s.affected = s.deps.resolve_affected_executions(change, configuration_id=CFG)
    # The signal names the stage whose result the change invalidates. The permit is a
    # ceiling, and ceilings are enforced at Constraints.
    signal = GovernanceInvalidationSignal(
        "permit-withdrawn", "Constraints", "charging permit withdrawn", "monitor:site-monitor",
        change.observation_ids, (), s.active.execution_id, CFG)
    s.assessment = s.cont.checkpoint(
        active_execution_id=s.active.active_execution_id, invalidation_signals=(signal,),
        trigger_ids=(change.change_id,), checkpoint_at=at(10))
    return s.assessment

def stop(s, control="cancel"):
    """8. Fence unused authority, issue the stop, dispatch it, and record what happened."""

```

```

aid, fence = s.active.active_execution_id, s.owner.owner_fence
# Fencing is the owner's act, and the runtime does not gate it: fence only when the
# checkpoint says the execution's authority must be fenced.
s.invalidated = ()
if s.assessment.authority_fence_required:
    s.invalidated = s.ctl.fence_execution_authority(
        active_execution_id=aid, owner_id="host-1", owner_fence=fence,
        reason="charging permit withdrawn")
req = s.ctl.issue_control_request(
    assessment_id=s.assessment.assessment_id, active_execution_id=aid, owner_id="host-1",
    owner_fence=fence, requested_control=control, target_handle=s.job, created_at=at(10))
s.attempt = s.ctl.dispatch_control_request(
    control_request_id=req.control_request_id, owner_id="host-1", owner_fence=fence,
    adapter_registration_id=s.adapter.adapter_registration_id, target_handle=s.job,
    control_region_at_request="cancelable", capability_evidence_ref="charger-caps-1",
    mapping_identity="charger-map-1", dispatched_at=at(10))
outcome, evidence = s.charger.send(s.attempt.mapped_command, s.job) # the charger acts
s.enforcement = s.ctl.record_enforcement_outcome(
    control_attempt_id=s.attempt.control_attempt_id, outcome=outcome,
    controller_evidence_ref=f"{s.job}:ack", observed_control_region=s.charger.jobs[s.job],
    recorded_at=at(11), attestation_evidence=evidence)
return s.enforcement

if __name__ == "__main__":
    s = start_supervised_charge()
    a = withdraw_permit(s)
    e = stop(s)
    print("affected executions:", len(s.affected))
    print("posture:           ", a.continuation_posture,
          "| fence required:", a.authority_fence_required)
    status = s.rt.native_execution_authorization_status(s.auth.authorization_id)
    print("authority:         ", status, f"({len(s.invalidated)} unused authorization fenced)")
    print("control:           ", s.attempt.requested_control, "->", s.attempt.mapped_command)
    print("controller outcome: ", e.outcome, "| attestation:", e.attestation_status)
    print("charging job:       ", s.charger.jobs[s.job])

```

Read the numbered comments in order.

- **1–2.** The episode is opened through a CanonicalRuntimeBridge rather than on the runtime directly, and the execution is registered as active in a SQLiteSupervisoryStore. Registration needs a native execution authorization: it is the runtime's record of this execution's authority, and it is what the fence in step 8 withdraws. The store here is in memory. In a deployment it is a file, so that the records survive a restart.
- **3.** The monitor is registered as a source for one kind of fact, charge_permit, about one subject, in one configuration. Its first report, granted, is admitted. Then the execution declares that it depends on that fact. Dependencies are never discovered: an execution is affected only by the facts it declares (Reference 16.5).
- **4.** One supervisory owner holds the execution, with a fence number that rises each time ownership changes hands, so that a stale owner cannot issue a stop. The adapter states which controls the charger supports and the charger's own command for each: cancel becomes stop_charge.
- **5.** The host starts the charge by its own means, as in step 5, and the episode stays open while the charge runs.
- **6–7.** The withdrawal is reported, admitted, and fanned out. For each affected execution the host checkpoints with a GovernanceInvalidationSignal naming the stage whose result the change invalidates. A withdrawn permit invalidates the result of Constraints, where ceilings are enforced (Table 2.2). The runtime does not infer that the value withdrawn invalidates Constraints. The host supplies the signal naming the stage whose result is no longer valid, and the runtime governs continuation from it.

- **8.** Because the checkpoint requires it, the owner fences the execution's unused authority, then issues a cancel, dispatches it through the adapter, lets the charger act, and records the charger's report.

Listing 6.2 — Output of step 6

```
affected executions: 1
posture:             reauthorization_required | fence required: True
authority:            invalidated (1 unused authorization fenced)
control:              cancel -> stop_charge
controller outcome:   completed | attestation: verified
charging job:         stopped
```

6.4 Reading the Output

The posture `reauthorization_required` is the runtime's decision. The Constraints result the execution was licensed under is no longer valid, so the execution may not continue without being governed again. The fence withdrew the one authorization that had not been used. The cancel was translated to the charger's `stop_charge`, the charger reports completed, and its attestation verified against the trust configuration. Only then does the job show stopped.

Notice what the runtime did not do. It did not stop the charger, and it did not decide what to do next. Governing the sensor again is the host's next step: a new decision cycle in which the world adapter reports that recharging breaks the permit rule. The runtime will then select from what remains, or stop with a stated reason (Section 4.2).

One boundary remains. The charger's report that it completed the stop is enforcement evidence, not proof of what the world now looks like. In this example the stand-in charger's job state is all there is, so the chapter ends there. In a deployment, where the realized effect matters, the host reconciles it separately, from authoritative evidence, before the next Layer-1 transition, and an unknown or timed-out effect stays unknown. Chapter 12 of the Guide shows that step (Reference 16.8).

The decision also limits the stop. A `reauthorization_required` decision permits hold, pause, cancel, or `reauthorization_required`, but not terminate, which only an `abort_return` or `emergency_return` decision permits. A decision of continue permits no stop at all, so a monitor cannot halt a healthy action. The tests in Section 6.6 check both.

6.5 When the Withdrawal Arrives Matters

A withdrawal can arrive at three different moments, and build 167 handles them differently. Know which one your application is in.

Table 6.2 — Three moments a withdrawal can arrive

When	What build 167 does	What you must do
Before the action starts	The fence invalidates the unused authorization. A native call attempted afterward through <code>invoke_registered_native</code> is refused	Check the authorization's status before starting work by your own means
While the host carries out the action by its own means (this chapter)	The full chain of Table 6.1: checkpoint, fence, governed stop request, recorded outcome	Send the command to your controller, and record what it reports

When	What build 167 does	What you must do
After a native call has been entered through <code>invoke_registered_native</code>	The authorization was consumed at call entry. From then on, every checkpoint of that execution is refused with <code>CanonicalResolutionError</code> , so no governed stop request can be issued	Stop the call through your own controller, outside the runtime's records. Better: do not run work you may need to stop inside a native call

The third row is an implementation boundary of v2.1.167: the runtime does not withdraw authority that has already been used, and it fails closed rather than govern an execution it can no longer resolve. But it means that once your agent's work is running inside a native call, the runtime cannot issue a governed stop for it. For work you may need to stop, follow this chapter's pattern: start the work by your own means and keep the episode open while it runs.

Two more limits apply in every case. A stop that arrives after the work is done is recorded as `too_late`, and a timeout is never recorded as no effect. And an effect that cannot be interrupted, such as a payment sent or a message delivered, is not undone by any stop request (Reference 16.8; 16.12).

6.6 Test It

These seven tests check that the withdrawal stops the charge, that the fence withdraws only unused authority, that the decision limits which stop may be requested, that a report from an unregistered source changes nothing, that a late stop is recorded as too late, that progress and completion are reported through checkpoints, and that a healthy execution cannot be stopped.

Listing 6.3 — `test_supervise.py`

```

"""Tests for step 6. Run: PVPP_RUNTIME_PATH=<runtime folder> python3 -m pytest -q"""

import pytest
from pvpp_runtime import ExecutionObservation
from step6_supervise import start_supervised_charge, withdraw_permit, stop, report, at

def test_withdrawn_permit_stops_the_running_charge():
    s = start_supervised_charge()
    a = withdraw_permit(s)
    assert s.affected == (s.active.active_execution_id,)
    assert a.continuation_posture == "reauthorization_required" and a.authority_fence_required
    e = stop(s)
    assert s.attempt.mapped_command == "stop_charge"
    assert e.outcome == "completed" and e.attestation_status == "verified"
    assert s.charger.jobs[s.job] == "stopped"

def test_fence_invalidates_authority_not_yet_used():
    s = start_supervised_charge()
    withdraw_permit(s)
    stop(s)
    assert s.invalidated == (s.auth.authorization_id,)
    assert s.rt.native_execution_authorization_status(s.auth.authorization_id) == "invalidated"

def test_posture_limits_the_stop_the_host_may_request():
    s = start_supervised_charge()
    withdraw_permit(s)
    with pytest.raises(ValueError, match="control_not_permitted_by_continuation_posture"):

```

```

        stop(s, control="terminate")                # terminate needs abort or emergency return

def test_report_from_an_unregistered_source_changes_nothing():
    s = start_supervised_charge()
    assert report(s, "rumor-1", "withdrawn", at(5), source="unknown-tool") is None
    view = s.obs.read_current_fact_view(configuration_id="battery-site", subject_id="sensor-A",
                                         fact_kind="charge_permit", fact_id="site")
    assert view.value == "granted"

def test_a_stop_that_arrives_too_late_is_recorded_as_too_late():
    s = start_supervised_charge()
    withdraw_permit(s)
    s.charger.jobs[s.job] = "finished"             # the job ended before the stop arrived
    assert stop(s).outcome == "too_late"

def test_progress_and_completion_are_reported_through_checkpoints():
    s = start_supervised_charge()
    aid = s.active.active_execution_id
    tick = s.cont.checkpoint(active_execution_id=aid, checkpoint_at=at(5),
                             observation=ExecutionObservation("charge-tick-1"))
    assert tick.continuation_posture == "continue" and tick.epsilon_status == "active"
    done = s.cont.checkpoint(active_execution_id=aid, checkpoint_at=at(20),
                             observation=ExecutionObservation("charge-done", completed=True))
    assert done.continuation_posture == "continue" and done.epsilon_status == "completed"

def test_a_healthy_execution_cannot_be_stopped_by_governance():
    s = start_supervised_charge()
    s.assessment = s.cont.checkpoint(active_execution_id=s.active.active_execution_id,
                                     checkpoint_at=at(5),
                                     observation=ExecutionObservation("charge-tick-1"))
    with pytest.raises(ValueError, match="control_not_permitted_by_continuation_posture"):
        stop(s)                                     # posture "continue" permits no stop at all
    assert s.rt.native_execution_authorization_status(s.auth.authorization_id) == "issued"

```

Listing 6.4 — Running all of the tutorial's tests

```

$ PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider
.....                                     [100%]
13 passed

```

6.7 What You Just Used

Table 6.3 — The supervision calls in step 6 and where the Reference documents them

Purpose	Calls	Reference
Trust	HostTrustProvider	16.2
Store	SQLiteSupervisoryStore	16.10
Register the execution	CanonicalRuntimeBridge: instantiate_execution, issue_native_execution_authorization, resolve_canonical_execution, register_active_execution; ExecutionBindingRegistry; ExecutionBindingIdentity	16.3; 11.5–11.6
Monitor	ObservationService: register_source, submit_observation, assess_observation, commit_admission_change, read_current_fact_view; SourceRegistration; ExternalObservation	16.4

Purpose	Calls	Reference
Dependencies	DependencyService: register_dependency_binding, resolve_affected_executions	16.5
Checkpoint	ContinuationService.checkpoint, with an ExecutionObservation or a GovernanceInvalidationSignal	16.6; 11.4; 12.2
Ownership and stop	ControlService: register_supervisory_owner, acquire_ownership, register_adapter, fence_execution_authority, issue_control_request, dispatch_control_request, record_enforcement_outcome	16.7
Status	native_execution_authorization_execution_id; native_execution_authorization_status	11.6

All of these are labeled PUBLIC / SUPPORTED in the Reference except four that are PUBLIC / ADVANCED: SQLiteSupervisoryStore, GovernanceInvalidationSignal, and the two native-authorization status calls.

Two entries of the bridge are guarded once an execution is supervised, and the tutorial uses neither: invoke_authorized_native and direct_advance_execution through the bridge. They are LEGACY / COMPATIBILITY guards, kept for executions that never entered supervision (Section 4.1; Reference 16.3).

The runtime decides that an action may not continue. Your controller stops it, and the runtime records what your controller reports.

Chapter 7 — Seven Things the Runtime Will Not Do for You

The runtime enforces its stages strictly. Some duties stay with the host anyway, because the runtime cannot fully enforce them. Depending on the duty, breaking one may raise an error, fail closed, or produce a result that looks valid and is not (Reference 15.3; 16.12). The Reference lists every such duty. These seven matter most for a first application: four for deciding and acting, and three more once you supervise an action while it runs. One more, for re-entry, follows the table.

Table 7.1 — Host duties for a first application

Duty	What goes wrong if you skip it	Habit	Reference
1. Carry out each decision at most once	One decision can yield several valid licenses, and each can be executed	Record which decisions you have executed; check before licensing	11.2
2. Keep episode ids unique; advance only the latest episode; treat the first terminal result as final	A reused id silently replaces the earlier episode's record, and an older episode object can still be advanced to a second, different result	Generate ids; discard episode objects once replaced; stop at the first terminal result	11.3; 11.4
3. Supply projections through a projection service	The older adapter method still works but skips the runtime's projection checks	Pass <code>projection_service=</code> to <code>PVPPRuntime</code> , as the tutorial does	10.3
4. Keep adapter hooks free of side effects	The runtime checks only its own records. A hook that changes state, or answers differently when asked twice, corrupts the decision unseen	Hooks read and compute; only the transition service changes state	10.10
5. Checkpoint every execution a fact change affects	Fan-out finds them, but nothing is governed until your code checkpoints each one. An execution you skip runs on under a decision that is no longer valid	Every time a change is committed, checkpoint every execution <code>resolve_affected_executions</code> returns	16.5; 16.6

Duty	What goes wrong if you skip it	Habit	Reference
6. Treat a stop request as a request, and enforcement as enforcement	A request, even a dispatched one, says nothing about whether the action stopped, and a controller's "completed" describes what it did, not the effect. <code>too_late</code> , <code>timed_out</code> , <code>unknown</code> , or an unverified attestation means the effect may have happened	Act on the enforcement record for the stop; reconcile the realized effect separately before the next state change; never record a timeout as no effect	16.7; 16.8
7. Do not run work you may need to stop inside a native call	Once <code>invoke_registered_native</code> has entered the call, every checkpoint of that execution is refused, so the runtime cannot issue a governed stop	Carry out stoppable work by your own means with the episode open, as in Chapter 6	16.3; 16.12

The tutorial meets the first four, and Chapter 6 meets the last three.

Advanced: re-entry only. After re-entry at Π completeness or earlier, check the joint-recovery result before licensing. Because of defect K2, those re-entry calls do not stop when the recovery obligations already active cannot all be met, and their decision can still be licensed and executed. If the joint-recovery record is runtime-verified and infeasible, treat the pass as stopped (Reference 12.5; 15.2). The tutorial uses no re-entry, so this never arises in it; it is here because the consequence is serious. Build 167 does not change it: its decision core is build 141's.

Two more habits protect everything else. Verify the runtime's hashes before relying on any result (Section 3.2). And remember what no check can do: the runtime cannot tell whether your facts are true. A projection that is well formed but wrong passes every check. Your tests are what catch it (Section 5.7).

Chapter 8 — Developing with AI

The three volumes of this documentation run to more than 500 pages, and the runtime exposes more than 500 names. No one should have to hold all of that in their head to write a small application. An AI assistant can hold it for you: it can read the whole Reference, find the call you need and what it checks, and draft the host code. It can also mislead you with complete confidence. This chapter is about getting the first without the second.

Let the AI read and draft. Keep the decisions, the facts, and the verification for yourself.

8.1 Why AI Helps Here

- **Lookup at the scale of the Reference.** “Which call builds an execution license, what does it refuse, and what must I do that it does not check?” is a question the Reference answers exactly, in three different sections. An assistant with the Reference attached finds and joins them in seconds.
- **Modeling by interview.** Most of the work in a Productive Value–Productive Power (PV-PP) application is deciding what the domains are, where they fail, and what each option does to them. The Guide is built for this: every chapter ends with a prompt that has an AI interview you and fill in a definition packet (Guide 1.12, Appendix A).
- **Host code with a fixed shape.** A registry, a world adapter, a projection service, a decision step, and an acting step (Chapter 5) have the same shape in every application. Drafting them from a packet is exactly what an assistant does well, and the tutorial gives it a correct pattern to follow.
- **Tests from expectations.** You know what your application should decide in each situation. An assistant can turn that list into tests quickly, which makes the decisions checkable rather than hoped for.

8.2 Where AI Goes Wrong with the PV-PP Runtime

These are not hypothetical. Each comes from the Reference’s own audit of the runtime. The Guard column names the numbered rules of Listing 8.1 (Section 8.4) that counter each trap.

Table 8.1 — Known traps for an AI assistant

Trap	Why it happens	Guard (Listing 8.1)
It suggests legacy entry points such as <code>assess</code>, <code>decide</code>, or <code>evaluate_policy_space</code>	The legacy entries remain in the runtime for compatibility, and the runtime keeps tests that prove they still work. Most of the shipped example scripts use them by design, as the small worlds those tests run on (Reference 15.4, E.2). An assistant that learns from the repository, or remembers an earlier version, can mistake them for patterns to follow	Rules 2 and 3

Trap	Why it happens	Guard (Listing 8.1)
It projects through the adapter's <code>project_policy_record</code> method	It still works and appears in older code, but receives none of the runtime's projection checks (Reference 10.3)	Rule 4
It writes a projection that looks right but is not	A well-formed projection record passes every runtime check even if its numbers are wrong or quietly optimistic; only your domain knowledge and your tests catch it (Guide 9.12)	Rule 8; read the projection code yourself
It trusts the runtime to enforce everything	Some duties stay with the host because the runtime cannot check them (Reference 15.3; Chapter 7)	Rule 6
It “fixes” a problem by editing the runtime	Editing is often the shortest path to making code run. The result is no longer v2.1.167, and nothing in the Reference applies to it	Rule 1; the hash check (Section 3.2)
It says the runtime stops a running action	The supervision layer is easy to describe as cancelling calls. It decides and records a stop request; your controller stops the action (Chapter 6)	Rule 9
It calls <code>invoke_authorized_native</code> or the bridge's <code>advance_execution</code> on a supervised execution	Both still exist on the bridge for executions outside supervision; once an execution is registered they are refused (Reference 16.3)	Rules 2 and 9
It answers from memory	Earlier runtime generations (v0.70, v0.131) had different surfaces, and build 167 was developed under the names Runtime 2.2 and v0.167; an assistant may blend them	Rule 7; attach the Reference

8.3 How to Work

A workflow that keeps the assistant useful and you in control:

- **1. Model first, in words.** Use Prompt 1 with the Guide attached. Answer its questions; correct its assumptions. The packet it produces is yours: every threshold and every projected consequence is a claim you are making about your domain.
- **2. Write down what the application should decide.** Before any code exists, list situations and the choice you expect in each, as Chapter 5 did for 9.0 and 5.0 units. This list becomes the tests.
- **3. Generate the code.** Use Prompt 2 with the Reference and the tutorial files attached. Check its call table against Appendix A of the Reference, and Table 6.3 for supervision calls, before reading further.
- **4. Run it against the verified runtime.** Run the hash check (Section 3.2), then each step file, then the tests. If a decision surprises you, find out whether your model or the code is wrong before changing either.

- **5. Review adversarially.** Use Prompt 3, ideally in a fresh conversation or with a different assistant, so the review is not the author grading its own work.

8.4 The Rules to Paste into Every Prompt

The rules in Listing 8.1 turn the traps of Table 8.1 into instructions. Paste them into every conversation about PV-PP code. They are deliberately blunt: an assistant follows an explicit list far more reliably than a general request to “follow the Reference”.

Listings 8.1 to 8.4 are complete as printed. The same text is supplied as a plain-text file, `getting-started/ai-rules-and-prompts.txt` in the repository, only so that it can be pasted into an assistant without picking up formatting from this document.

Listing 8.1 — Rules for AI-assisted PV-PP development

RULES FOR THIS WORK (PV-PP runtime v2.1.167)

1. The runtime is frozen. Never edit, patch, or monkeypatch anything in runtime-v2.1.167 or `pvpp_runtime`, and never use a name that begins with an underscore. My code lives in its own folder.
2. Use entries labeled PUBLIC / SUPPORTED in Appendix A of the attached Programmer's Reference. Use a PUBLIC / ADVANCED entry only when you say why. Never use a LEGACY / COMPATIBILITY entry: `assess`, `decide`, `evaluate_policy_space`, `evaluate_policy_sets`, `evaluate_recovery_policy_space`, `evaluate_constraints`, `evaluate_framed_policy_space`, `evaluate_proposal`, `validate_domain_framing`, `build_execution_license`, `record_execution`, `record_layer1_execution_commit`, `register_domain_relation`, `register_dependency_requirement`, `ExecutionBindingRegistry.invoke`, `ExecutionBindingRegistry.build_layer1_handoff`, or the world adapter's `project_policy_record` method.
3. Do not copy patterns from the runtime's `tiny_*` examples. They exist for the legacy compatibility surface and its tests, kept by design, not as models for new code.
4. Supply projections through a `PolicyProjectionService` passed as `projection_service=`. Each record echoes the request's `state_id`, `state time`, `candidate_mode`, `projection horizon`, and `projection_input_trace`, names a `model_version`, and marks horizons at the limit as right-censored.
5. Form perception with `perceived_decision_state_from_actual`; `perceive` returns its argument unchanged.
6. Host duties the runtime does not check: execute each decision at most once; keep episode ids unique and advance only the latest episode; treat the first terminal result as final; after re-entry at Pi Completeness or earlier, never license a decision whose joint-recovery result is runtime-verified infeasible; change state only through the Layer-1 transition service; checkpoint every execution a committed fact change affects; act on enforcement outcomes, not on control requests.
7. For every runtime call you use, cite the Reference section that documents it. If the Reference does not answer a question, say so. Do not guess, and do not rely on what you remember of earlier runtime versions.
8. Write tests with the code: at least one for each decision the code relies on.
9. For actions supervised while they run (`pvpp_runtime.supervision`): use its services, never the store's tables or private helpers; treat a monitor report as evidence, never as a decision; never say the runtime stops a process: it records a governed stop request that my controller carries out; and do not put work that may need to be stopped inside `invoke_registered_native`.

8.5 Three Prompts

Each prompt below is meant to be used as written, with the rules of Listing 8.1 pasted above it and the named documents attached. Replace the text in angle brackets.

Listing 8.2 — Prompt 1: model my situation

PROMPT 1: MODEL MY SITUATION

Attach: the Programmer's Guide. Paste: the rules above.

I want to model the situation below with the PV-PP framework. Interview me, one question at a time, until you can fill in a model definition packet with these parts: what must be preserved; the domains, each with what it measures and the level at which it fails; the actions, including "steady" (what happens if nothing new is done) and how each action changes each domain per period; the recovery routes; the hard rules; the options (graph

transformations) available from the current state, each with its family; and the horizon over which options should be compared.

Use the Guide's Chapter 3 prompt (Guide A.3) as your checklist. Mark every number I have not given you as an assumption, and list the assumptions at the end. Do not write code yet.

My situation: <describe it in a few plain sentences>

Listing 8.3 — Prompt 2: write the host code

PROMPT 2: WRITE THE HOST CODE

Attach: the Programmer's Reference; the Getting Started tutorial files (step1 to step6, test_battery.py, and test_supervise.py); the packet from Prompt 1. Paste: the rules above.

Write host code for the attached model, following the structure of the tutorial: a registry, a world adapter, a projection service, a decision step, and an acting step with a Layer-1 transition service, and, if any action runs over time while facts can change, a supervision step in the shape of step 6; each in its own file that runs on its own and prints what it built. Then write a test file with one test for each decision the model is meant to make, and one showing that the Sigma order only breaks ties.

Before the code, give a table of every runtime call you use, its label from Reference Appendix A, and the Reference section that documents it. After the code, list the scenarios you tested and the outcome you expect in each, so I can check them against my own expectations before I run anything.

Listing 8.4 — Prompt 3: review this host code

PROMPT 3: REVIEW THIS HOST CODE

Attach: the Programmer's Reference; my host code and tests. Paste: the rules above.

Review my host code against the Programmer's Reference as a hostile reviewer. For each finding give the file and line, the rule or Reference section it breaks, why it matters, and a corrected version. Check in particular: any legacy or internal entry; any edit to the runtime; projections that do not echo the request or that report a domain as safe beyond what the model supports; perception that invents facts; any decision executed more than once or episode advanced out of order; state changed outside the Layer-1 service; decisions the tests do not cover; and, if the code supervises actions, any affected execution never checkpointed, any stop request treated as a completed stop, and any stoppable work run inside invoke_registered_native. Do not rewrite code that has no finding.

8.6 What Stays with You

An assistant can find, draft, and check. It cannot know your domain, and it cannot vouch for itself. Four things stay with you, however good the assistant:

- The facts: thresholds, rates, and projected consequences are claims about your world. Read them in the code as carefully as you gave them in the interview.
- The expectations: decide what the application should do before you see what it does.
- The verification: the hash check, the step outputs, and the tests, run by you against the frozen runtime.
- The judgment when the two disagree: a surprising decision is information. Sometimes the runtime has found something your intuition missed; sometimes the model is wrong. Find out which before changing anything.

Chapter 9 — Where to Go Next

This book gets a first application running. The other two volumes take over from here.

Table 9.1 — Which book answers which question

If you are asking	Go to
How should I model my situation: its domains, thresholds, and options?	Guide Chapters 3 and 5; the interview prompts in Guide Appendix A
Why does the runtime work this way? What is a stage for?	Guide Chapters 2 and 6
How do I build a larger application, with several domains, stale evidence, and failures?	Guide Chapters 6–9, which grow a coffee-vending machine step by step, and Guide Chapter 10
What exactly does this call accept, return, and refuse?	Reference, the section Appendix A names for it
Why did the runtime refuse?	Reference 14.11, then the stage’s contract in Reference Chapter 3
What must my code guarantee that the runtime does not check?	Reference 15.3 and 16.12; Chapter 7 of this book
What changed when the facts changed, and what must run again?	Reference Chapter 12
How do I stop an action when a monitor withdraws its authority?	Chapter 6 of this book; Guide Chapter 12; Reference Chapter 16
What may a new actor inherit from the one that created it?	Section 1.4 of this book
Is this copy of the runtime really v2.1.167?	Section 3.2 of this book; Reference 13.7
Which versions of the runtime, framework, and books go together?	Reference Appendix D
Where does this rule come from in the framework?	Reference Appendix F and the compendia it cites

9.1 Code to Read Next

- Reference Appendix B: the same five pieces as the tutorial, in reference form, plus re-entry after a change and a supervised execution.
- The Guide’s programs, built on a coffee-vending machine, from one domain to several.
- Of the examples shipped with the runtime, `examples/authority_bound_callthrough_v0130.py`; `examples/minimal_battery_service` only with the sign erratum in Section 4.1. Not the `tiny_*` examples.
- The runtime’s own supervision tests, `tests/test_r22_*.py` in the runtime folder, which exercise every link of Table 6.1, including races, crashes, and recovery. Read them to see behavior. Do not copy their fixture credentials or their direct use of the store.

9.2 Working with an Assistant

For anything larger than the tutorial, use the workflow of Chapter 8: model in words first, write down what the application should decide, generate the code with the Reference attached, run it against the verified runtime, and have it reviewed by a second assistant. The rules in Listing 8.1 apply to every step.